

Practical Lecture 2

Hebb Network for Logic Functions

MATLAB Laboratory



The Islamic University in Najaf Al-Ashraf
College of Technical Engineering
Department of Computer Techniques Engineering
Instructor Jaafar Al-Kafnawiz



Train Hebbian networks for AND and OR using bipolar inputs and bipolar target values.

Supervised Hebbian learning adds the product of the input and target to each weight. Bipolar encoding maps logical 0 to -1 and logical 1 to +1. The final sign of the net input gives the predicted logical class.

Expected learning outcome: students connect the mathematical rule with the values printed by the MATLAB program.



1

Convert binary inputs to bipolar values.

2

Initialize the weights and bias to zero.

3

Accumulate x times target for every pattern.

4

Calculate the net input and classify its sign.



```
% Practical 2: Hebb network for AND and OR
clear; clc;

Xbinary = [0 0; 0 1; 1 0; 1 1];
X = 2 * Xbinary - 1;           % Convert 0 to -1 and 1 to +1

gateNames = {'AND','OR'};
targets = [-1 -1 -1 1;        % AND target
           -1  1  1  1];      % OR target

for g = 1:numel(gateNames)
    target = targets(g,:);
    [weights, bias] = trainHebb(X, target);
    net = X * weights + bias;
    prediction = ones(size(net));
    prediction(net < 0) = -1;
end
```



```
fprintf('\n%s gate\n', gateNames{g});
fprintf('Weights = [%1f %1f], bias = %1f\n', ...
    weights(1), weights(2), bias);
disp(table(Xbinary(:,1), Xbinary(:,2), target, prediction, ...
    'VariableNames', {'x1', 'x2', 'target', 'prediction'}));
end

function [weights, bias] = trainHebb(X, target)
% Hebbian supervised update: delta_w = x * target.
weights = zeros(size(X,2),1);
bias = 0;
for i = 1:size(X,1)
    weights = weights + X(i,:) * target(i);
    bias = bias + target(i);
end
end
```



MATLAB Command Window

```
>> Practical_02_Hebb_Logic_MATLAB
```

```
AND: weights = [2 2], bias = -2  
prediction = [-1 -1 -1 1]
```

```
OR: weights = [2 2], bias = 2  
prediction = [-1 1 1 1]
```